Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

Receptes oldal

Készítette: Erdei Gergő János
Kiss Bence
Szabó Balázs

2026.

Tartalomjegyzék

Tartalomjegyzék ..... Hiba! Nincs megadva könyvjelzőnév.
1. Bevezetés ..... Hiba! Nincs megadva könyvjelzőnév.
1.1. Témaválasztás indoklása .....**Hiba! Nincs megadva könyvjelzőnév.**
1.2. Kitűzött célok.....**Hiba! Nincs megadva könyvjelzőnév.**
1.3. Technológiai választások .....**Hiba! Nincs megadva könyvjelzőnév.**
1.4. Csapatmunka megvalósítása 5
1.5. Projekt futtatása 5
2. Backend fejlesztői dokumentáció..... Hiba! Nincs megadva könyvjelzőnév.
2.1. Fejlesztői környezet.....**Hiba! Nincs megadva könyvjelzőnév.**
2.2. Projekt struktúra.....**Hiba! Nincs megadva könyvjelzőnév.**
2.3. Adatbázis modell .....**Hiba! Nincs megadva könyvjelzőnév.**
2.4. API Endpointok .....**Hiba! Nincs megadva könyvjelzőnév.**
2.5. Kiemelt kódrészletek .....**Hiba! Nincs megadva könyvjelzőnév.**
2.5.1. JWT Token generálás és validáció.....**Hiba! Nincs megadva könyvjelzőnév.**
2.5.2. Kép feltöltés ImageKit-tel .....**Hiba! Nincs megadva könyvjelzőnév.**
2.5.3. Keresési funkció implementáció .....**Hiba! Nincs megadva könyvjelzőnév.**
2.6. Tesztelés és hibakezelés .....**Hiba! Nincs megadva könyvjelzőnév.**
2.6.1. Scalar API dokumentáció .....**Hiba! Nincs megadva könyvjelzőnév.**
2.6.2. Előforduló hibák és megoldások .....**Hiba! Nincs megadva könyvjelzőnév.**
2.7. Localhost konfiguráció.....**Hiba! Nincs megadva könyvjelzőnév.**
2.7.1. Backend konfiguráció.....**Hiba! Nincs megadva könyvjelzőnév.**
2.7.2. Frontend konfiguráció .....**Hiba! Nincs megadva könyvjelzőnév.**
2.7.3. WPF Admin Panel konfiguráció.....**Hiba! Nincs megadva könyvjelzőnév.**
3. Frontend fejlesztői dokumentáció..... 13
3.1. Fejlesztői környezet..... 13
3.2. Használt technológiák 13
3.3. Projekt struktúra..... 14
3.4. Routing (oldalkezelés) 15
3.5. Globális állapotkezelés (React Context) 16
3.5.1. AuthenticatedUserContextProvider 16
3.5.2. MobileContextProvider..... 17
3.6. Oldalak és komponenseik részletesen 18
3.6.1. Home oldal..... 18
3.6.2. Navbar komponens..... 19
3.6.3. ForYou oldal 19
3.6.4. Login oldal..... 21
3.6.5. Signup oldal..... 22
3.6.6. ContentUpload – recept feltöltő oldal..... 23
3.6.7. ErrorPage 24
3.7. API kommunikáció..... 24

3.8. Állapotkezelés (TanStack Query) ... 25
3.9. Autentikáció kezelése ... 26
3.10. Stíluskezelés (CSS Modules) ... 26
3.11. TypeScript típusdefiníciók ... 27
3.12. Hibakezelés és felhasználói visszajelzés ... 27
3.13. Felhasználói élmény (UI/UX) ... 28
3.14. Weboldal elérése ... 28
4. Összegzés ... Hiba! Nincs megadva könyvjelzőnév.
4.1. Továbbfejlesztési lehetőségek ... **Hiba! Nincs megadva könyvjelzőnév.**
5. Melléklet - Képernyőképek ... Hiba! Nincs megadva könyvjelzőnév.
5.1. Fejlesztői környezet ... **Hiba! Nincs megadva könyvjelzőnév.**
Solution Explorer ... **Hiba! Nincs megadva könyvjelzőnév.**
Program.cs fájl ... **Hiba! Nincs megadva könyvjelzőnév.**
5.2. Adatbázis diagram ... 32
5.3. API dokumentáció és tesztelés ... **Hiba! Nincs megadva könyvjelzőnév.**
Scalar API dokumentáció főoldal ... **Hiba! Nincs megadva könyvjelzőnév.**
GET /api/Recept endpoint tesztelés Scalar-ban ... **Hiba! Nincs megadva könyvjelzőnév.**
POST /api/Auth/login sikeres válasz ... **Hiba! Nincs megadva könyvjelzőnév.**
Authorization Bearer token beállítása ... **Hiba! Nincs megadva könyvjelzőnév.**
5.4. Kódrészletek ... **Hiba! Nincs megadva könyvjelzőnév.**
AuthController.cs - JWT token generálás ... **Hiba! Nincs megadva könyvjelzőnév.**
ReceptController.cs - ImageKit kép feltöltés ... **Hiba! Nincs megadva könyvjelzőnév.**
UserDbContext.cs - Entity konfiguráció ... **Hiba! Nincs megadva könyvjelzőnév.**
6. Irodalomjegyzék ... Hiba! Nincs megadva könyvjelzőnév.

1. Bevezetés

1.1. Témaválasztás indoklása

A receptes weboldalak népszerűsége az utóbbi években jelentősen megnőtt, hiszen egyre többen keresnek praktikus, gyors megoldásokat a mindennapi főzéshez. A projektünk célja egy modern, felhasználóbarát recept megosztó platform létrehozása, ahol a felhasználók saját receptjeiket oszthatják meg, kereshetnek különböző ételek között, és interakcióba léphetnek más főzés iránt érdeklődőkkel.

A téma választását több tényező is indokolta:

- Gyakorlati hasznosság: A mindennapi életben közvetlenül alkalmazható alkalmazás
- Technológiai kihívás: Lehetőséget biztosít modern webfejlesztési technológiák alkalmazására
- Full-stack fejlesztés: Mind a frontend, mind a backend oldal komplex fejlesztést igényel
- Felhasználói élmény: A UX/UI tervezés fontosságának demonstrálása

1.2. Kitűzött célok

A projekt során a következő főbb célokat tűztük ki magunk elé:

- **RESTful API készítése:** Modern ASP.NET Core backend API fejlesztése, amely biztonságos és skálázható adatkezelést biztosít
- **Autentikáció és jogosultságkezelés:** JWT token alapú azonosítás implementálása, felhasználói szerepkörök (User/Admin) kezelése
- **Képkezelés:** Receptekhez és profilképekhez tartozó képek feltöltése, tárolása és kezelése cloud alapú szolgáltatással
- **Keresés és szűrés:** Hatékony keresési funkciók implementálása név és a megadott címkék alapján
- **Interakciós funkciók:** Like rendszer, kommentelés és receptek mentése funkciók megvalósítása
- **Admin felület:** Desktop alapú WPF admin panel fejlesztése a tartalom moderálásához

1.3. Technológiai választások

A projekt fejlesztéséhez az alábbi technológiákat választottuk:

- **.NET 10.0 és ASP.NET Core:** Modern, nagy teljesítményű, cross-platform keretrendszer
- **Entity Framework Core:** Object-Relational Mapping (ORM) a hatékony adatbázis-kezeléshez
- **SQLite:** Könnyű, fájl alapú adatbázis, ideális fejlesztéshez és kisebb alkalmazásokhoz
- **JWT (JSON Web Token):** Biztonságos authentikációhoz és authorization-höz
- **ImageKit.io:** Cloud alapú képkezelési szolgáltatás optimalizálással és CDN támogatással
- **Scalar és Postman:** Interaktív API dokumentáció és tesztelési felület
- **React:** Komponens alapú UI fejlesztés
- **TypeScript:** a JavaScript statikus típusossággal rendelkező kibővítése
- **Git**: egyszerű és átlátható verziókövetés
- **Visual Studio és Visual Studio Core**: a csapat preferált és megszokott fejlesztő környezetei

1.4. Csapatmunka megvalósítása

Korábbi projekteken is dolgoztunk már együtt, ezért döntöttünk úgy, hogy a vizsgaremekekünket is együtt fogjuk megcsinálni. Nagyrészt az iskolában dolgoztunk és szóban kommunikáltunk, egyébként Messengeren. Ezen dokumentációt közösen írtuk.

- **Backend**: Kiss Bence
- **Frontend**: Erdei Gergő, Szabó Balázs

1.5. Projekt futtatása

A feladatkezelőben egy tetszőleges mappában a mappa elérési útjába kattintva "cmd" parancs beirásával felnyiló ablakba kell bemásolni jobb klikk lenyomásával: "git clone https://github.com/KBencee/Receptes-oldal---VIZSGAREMEK". A kód megtekintésére és futtatására, a Visual Studiot és Visual Studio Codeot ajánljuk, mivel mi is ezeket használtuk.
Útmutató a backendhez: https://youtu.be/n35eT_f3ws4?si=a64sLg2sQjav8CFz

2. Backend fejlesztői dokumentáció

A projekt ezen részét Kiss Bence készítette.

2.1. Fejlesztői környezet

IDE és eszközök:

- Visual Studio 2026 Community Edition
- .NET 10.0 SDK
- Git verziókezelés (GitHub repository)
- Postman / Scalar - API teszteléshez
- DB Browser for SQLite - adatbázis kezeléshez

NuGet csomagok:

- Microsoft.AspNetCore.Authentication.JwtBearer - JWT authentikáció
- Microsoft.EntityFrameworkCore.Sqlite - SQLite támogatás
- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Tools
- Imagekit - képkezelés
- BCrypt.Net-Next - jelszó hashelés
- Scalar.AspNetCore - API dokumentáció

2.2. Projekt struktúra

A backend projekt az alábbi mappastruktúrát követi:

- **Controllers/:** API endpointokat tartalmazó controller osztályok
- **Models/:** DTO (Data Transfer Object) modellek
- **Entities/:** Alap modellek/entitások
- **Data/:** DbContext és adatbázis konfigurációk
- **Migrations/:** Entity Framework migrációs fájlok
- **Services/:** Üzleti logika és külső szolgáltatások (pl. ImageKit)

Főbb fájlok:

- Program.cs - alkalmazás konfigurációja, middleware-ek, dependency injection
- appsettings.json - konfigurációs beállítások, connection string-ek

2.3. Adatbázis modell

Az alkalmazás SQLite adatbázist használ, amely a következő főbb entitásokat tartalmazza:

User (Felhasználó):

- Id (Guid) - elsődleges kulcs
- Username (string) - felhasználónév, egyedi
- PasswordHash (string) - BCrypt-tel hashelt jelszó
- Role (string) - szerepkör (User / Admin)
- RefreshToken, RefreshTokenExpiryTime - token frissítéshez
- ProfilKepUrl, ProfilKepFileId - profilkép adatok

Recept:

- Id (Guid) - elsődleges kulcs
- Nev (string) - recept neve
- Leiras (string) - részletes leírás
- Hozzavalok (string) - hozzávalók listája
- ElkeszitesiIdo (int) - elkészítési idő percben
- NehezsegiSzint (string) - könnyű / közepes / nehéz
- Likes (int) - like-ok száma
- UserId (Guid) - idegen kulcs a User táblára
- FeltoltveEkkor (DateTime) - feltöltés időpontja
- KepUrl, KepFileId - recept képe

További entitások:

- **Cimke:** Címkék (pl. Desszert, Vacsora, Vegetáriánus)
- **ReceptCimke:** Many-to-Many kapcsolótábla receptek és címkék között
- **ReceptKomment:** Kommentek a receptekhez
- **MentettRecept:** Felhasználók által mentett receptek
- **ReceptLike:** Like-ok nyilvántartása

Kapcsolatok:

- User → Recept (One-to-Many): egy felhasználó több receptet is feltölthet
- Recept ↔ Cimke (Many-to-Many): egy receptnek több címkéje lehet, egy címke több recepthez tartozhat
- User → ReceptKomment ← Recept: felhasználók kommenteket írhatnak receptekhez

2.4. API Endpointok

A backend RESTful API architektúrát követ, az alábbi főbb controller-ekkel:

AuthController (Authentikáció és felhasználókezelés):

- **POST /api/Auth/register** - új felhasználó regisztrációja
- **POST /api/Auth/login** - bejelentkezés (JWT token generálás)
- **POST /api/Auth/refresh** - access token frissítése
- **POST /api/Auth/logout** - kijelentkezés (token érvénytelenítés)
- **PUT /api/Auth/me/username** - saját felhasználónév módosítása
- **PUT /api/Auth/me/password** - saját jelszó módosítása
- **GET /api/Auth/admin/users** - összes felhasználó listázása (Admin)
- **PUT /api/Auth/admin/users/{id}** - felhasználó módosítása (Admin)
- **DELETE /api/Auth/admin/users/{id}** - felhasználó törlése (Admin)
- **GET /api/Auth/admin/stats** - statisztikák lekérdezése (Admin)

ReceptController (Receptek kezelése):

- **GET /api/Recept** - összes recept listázása
- **GET /api/Recept/{id}** - egy recept részletei
- **GET /api/Recept/my-recipes** - saját receptek
- **GET /api/Recept/search?query=...** - keresés név/címke/hozzávaló alapján
- **GET /api/Recept/by-tag/{cimkeId}** - receptek címke szerint
- **POST /api/Recept** - új recept létrehozása
- **POST /api/Recept/with-image** - recept létrehozása képpel
- **PUT /api/Recept/{id}** - recept módosítása
- **POST /api/Recept/{id}/add-image** - kép hozzáadása recepthez
- **PUT /api/Recept/{id}/update-image** - kép cseréje
- **DELETE /api/Recept/{id}** - recept törlése
- **POST /api/Recept/{id}/like** - recept like-olása
- **DELETE /api/Recept/{id}/like** - like visszavonása

További controller-ek:

- **UserController:** Felhasználói profil kezelés, profilkép feltöltés
- **KommentController:** Kommentek létrehozása, szerkesztése, törlése
- **MentettReceptekController:** Receptek mentése és lekérdezése
- **CimkekController:** Címkék kezelése

2.5. Kiemelt kódrészletek

Az alábbiakban bemutatom a backend néhány fontosabb és érdekesebb implementációját.

2.5.1. JWT Token generálás és validáció

Az alkalmazás JWT (JSON Web Token) alapú authentikációt használ. A token generálás a bejelentkezéskor történik:

[Program.cs - JWT konfiguráció]

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options =>    {        options.TokenValidationParameters = new
TokenValidationParameters        {            ValidateIssuerSigningKey = true,
IssuerSigningKey = new SymmetricSecurityKey(key),            ValidateIssuer = false,
ValidateAudience = false,            ClockSkew = TimeSpan.Zero        };    });
```

[AuthController.cs - Token generálás]

```
var claims = new[] {    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
new Claim(ClaimTypes.Name, user.Username),    new Claim(ClaimTypes.Role,
user.Role) }; var token = new JwtSecurityToken(    claims: claims,    expires:
DateTime.UtcNow.AddMinutes(15),    signingCredentials: new
SigningCredentials(key, SecurityAlgorithms.HmacSha256) );
```

A token 15 percig érvényes, utána refresh token használható új access token generálásához. Ez biztosítja a biztonságot, miközben a felhasználói élmény is jó marad.

2.5.2. Kép feltöltés ImageKit-tel

Az ImageKit.io cloud szolgáltatást használjuk a képek tárolására és optimalizálására. Ez automatikus képformátum-konverziót, átméretezést és CDN támogatást biztosít:

[ReceptController.cs - Kép feltöltés]

```
var imagekit = new ImagekitClient(publicKey, privateKey, urlEndpoint); using var
memoryStream = new MemoryStream(); await image.CopyToAsync(memoryStream);
var imageBytes = memoryStream.ToArray();  var uploadRequest = new
FileCreateRequest {    file = imageBytes,    fileName =
$"recept_{receptId}_{Guid.NewGuid()}.jpg" };  var result = await
imagekit.UploadAsync(uploadRequest); recept.KepUrl = result.url; recept.KepFileId =
result.fileId;
```

A képek törlése is ImageKit API-n keresztül történik, így nem halmozódnak fel használaton kívüli fájlok.

2.5.3. Keresési funkció implementáció

A keresési endpoint részleges egyezést (Contains) használ, így a felhasználók nem csak teljes szavakra kereshetnek:

[ReceptController.cs - Keresés]

```
var lowerQuery = query.ToLower(); var receptek = await _context.Receptek
.Include(r => r.ReceptCimkek).ThenInclude(rc => rc.Cimke)     .Include(r => r.User)
.Where(r =>          r.Nev.ToLower().Contains(lowerQuery) ||
r.Leiras.ToLower().Contains(lowerQuery) ||
r.Hozzavalok.ToLower().Contains(lowerQuery) ||         r.ReceptCimkek.Any(rc =>
rc.Cimke.CimkeNev.ToLower().Contains(lowerQuery))     )     .OrderByDescending(r =>
r.FeltoltveEkkor)     .ToListAsync();
```

A keresés a recept nevében, leírásában, hozzávalóiban és címkéiben is keres, így például a 'pal' keresőszó megtalálja a 'palacsinta' receptet is.

2.6. Tesztelés és hibakezelés

Az API teszteléséhez a Scalar interaktív dokumentációs felületet használtuk, amely a fejlesztés során automatikusan generálódik az OpenAPI specifikáció alapján.

2.6.1. Scalar API dokumentáció

A Scalar felület elérhető a http://localhost:7114/scalar/v1 címen, és lehetőséget biztosít:

- Összes endpoint megtekintésére kategóriák szerint csoportosítva
- Request és response sémák vizsgálatára
- Endpointok kipróbálására közvetlenül a böngészőből
- Authorization header beállítására JWT token-nel

2.6.2. Előforduló hibák és megoldások

A fejlesztés során több technikai problémába is ütköztünk:

1. CORS (Cross-Origin Resource Sharing) hibák:

- **Probléma:** A frontend (localhost:3000) nem tudott kapcsolódni a backend-hez (localhost:7114)
- **Megoldás:** CORS policy beállítása Program.cs-ben, amely engedélyezi a cross-origin requesteket

2. ImageKit kép törlés problémák:

- **Probléma:** Recept törlésekor a hozzá tartozó kép nem került törlésre ImageKit-ről
- **Megoldás:** Cascade delete implementálása, amely automatikusan törli a képet is

3. Entity Framework migrációk:

- **Probléma:** Entitások közötti kapcsolatok hibás konfigurációja miatt sikertelen migrációk
- **Megoldás:** Fluent API használata a DbContext OnModelCreating metódusában a pontos kapcsolatok definiálásához

4. JWT Token lejárat kezelése:

- **Probléma:** Felhasználók váratlanul kijelentkeztek 15 perc után
- **Megoldás:** Refresh token mechanizmus implementálása, automatikus token frissítés a frontend-en

2.7. Localhost konfiguráció

FONTOS: Az alkalmazás futtatásához localhost URL-eket kell használni, ne dev tunnel-t!

2.7.1. Backend konfiguráció

A backend alapértelmezetten a következő címen érhető el:

- **HTTPS:** https://localhost:7114
- **HTTP:** http://localhost:5000
- **Scalar API doc:** https://localhost:7114/scalar/v1

launchSettings.json beállítás:

"applicationUrl": "https://localhost:7114;http://localhost:5000"

2.7.2. Frontend konfiguráció

A React frontend-ben az API base URL-t környezeti változóban kell beállítani:

[.env fájl]

REACT_APP_API_URL=https://localhost:7114/api

[API service használat]

const API_BASE_URL = process.env.REACT_APP_API_URL; const response = await fetch(`${API_BASE_URL}/Recept`);

2.7.3. WPF Admin Panel konfiguráció

A WPF admin felületben az ApiService konstruktorában kell beállítani a localhost URL-t:

[ApiService.cs]

public ApiService() { _httpClient = new HttpClient { BaseAddress = new Uri("https://localhost:7114/api/") }; }

Fejlesztés vs. Production:

- **Fejlesztés:** localhost URL-ek használata
- **Production:** Környezeti változókból vagy konfigurációs fájlból olvasás

3. Frontend fejlesztői dokumentáció

A projekt ezen részét Erdei Gergő és Szabó Balázs készítette. A fejezet az alkalmazás felhasználói felületének teljes megvalósítását dokumentálja: az alkalmazott technológiákat, a projekt struktúráját, az egyes oldalakat és komponenseket, valamint a kiemelten fontos kódrészleteket.

3.1. Fejlesztői környezet

A frontend fejlesztés során modern webes technológiákat alkalmaztunk egy dinamikus, gyors és felhasználóbarát felület létrehozása érdekében.

Használt eszközök:

- Visual Studio Code – kódszerkesztő, kiegészítőkkel (Prettier, TypeScript, ESLint)
- Node.js és npm – csomagkezelés és build folyamat
- Git – verziókezelés (GitHub repository)
- Böngésző fejlesztői eszközök (Chrome DevTools) – hibakereséshez és reszponzív teszteléshez

3.2. Használt technológiák

A frontend fejlesztés során az alábbi főbb technológiákat és könyvtárakat alkalmaztuk:

- **React –** komponens alapú UI fejlesztés, deklaratív renderelés
- **TypeScript –** JavaScript kiegészítése statikus típusbiztonsággal, korai hibadetektáláshoz
- **React Router v7 –** kliens oldali routing és navigáció kezelésére, az oldal újratöltése nélkül
- **Axios –** HTTP kérések küldésére a backend API felé, interceptorokkal

- **TanStack Query (React Query) –** szerver állapot kezelésére, automatikus cache-eléssel és adatszinkronizációval
- **CSS Modules –** komponens szintű stíluskezelés, globális névütközések elkerülésére
- **Sonner –** toast értesítések, felugró visszajelző üzenetek megjelenítésére
- **Font Awesome –** ikon könyvtár (like, mentés, navigációs ikonok)

Ezek kombinációja lehetővé tette egy jól strukturált, karbantartható és skálázható frontend alkalmazás létrehozását.

3.3. Projekt struktúra

A frontend projekt logikailag elkülönített mappákra épül, amelyek segítik az átláthatóságot és a bővíthetőséget:

- **components/ –** újrafelhasználható UI komponensek (Navbar, RecipeCard, ForYouReel, Description stb.)
- **pages/ –** különálló oldalak, amelyek egy-egy route-nak felelnek meg (Home, ForYou, Login, Signup stb.)
- **services/ –** API hívások (publicAPI.ts, privateAPI.ts)
- **context/ –** globális állapot kezelésére szolgáló React Context-ek (AuthenticatedUserContextProvider, MobileContextProvider)
- **queryOptions/ –** TanStack Query lekérdezések konfigurációi (query key-ek, fetch függvények)
- **mutationOptions/ –** TanStack Query mutation-ök konfigurációi
- **css/ –** globális és modul szintű stílusok (.module.css fájlok)
- **types/ –** TypeScript típusdefiníciók (RecipeTypes.ts stb.)

Ez a struktúra támogatja a komponens alapú fejlesztést és az átláthatóságot, ami segíti a fejlesztést és a továbbfejlesztést egyaránt.

3.4. Routing (oldalkezelés)

Az alkalmazás kliens oldali navigációját a React Router v7 biztosítja. A routing konfigurációja az App.tsx fájlban található, ahol a BrowserRouter komponens alatt definiáljuk az összes útvonalat.

[App.tsx – routing konfiguráció]

```
<BrowserRouter>
  <Routes>
      <Route path='/'                   index element={<Home/>}/>
      <Route path='/own'              element={<Own/>}/>
      <Route path='/upload'          element={<ContentUpload_/>}/>
      <Route path='/saved'              element={<Saved/>}/>
      <Route path='/login'              element={<Login/>}/>
      <Route path='/signup'             element={<Signup/>}/>
      <Route path='/foryou/:id'         element={<ForYou/>}/>
      <Route path='/description/:id' element={<MobileDescription/>}/>
      <Route path='*'                element={<ErrorPage/>}/>
  </Routes>
</BrowserRouter>
```

A routing lehetővé teszi, hogy az oldal újratöltése nélkül navigáljon a felhasználó az alkalmazás különböző részei között. A /foryou/:id útvonal dinamikus paramétert (:id) tartalmaz, amellyel az adott recept azonosítóját adjuk át az URL-ben. A * útvonal az összes nem definiált útvonalat kezeli, hibás URL esetén az ErrorPage-et rendereli.

Útvonal	Oldal / Leírás
`/`	Home – főoldal, receptek kártyás listázása keresővel
`/foryou/:id`	ForYou – teljes képernyős recept néző, asztali és mobil nézettel

`/login`	Login – bejelentkezési form
`/signup`	Signup – regisztrációs form
`/upload`	ContentUpload – recept feltöltő oldal (bejelentkezés szükséges)
`/own`	Own – bejelentkezett felhasználó saját receptjei
`/saved`	Saved – bejelentkezett felhasználó elmentett receptjei
`/description/:id`	MobileDescription – mobil nézeten a recept részletei
`*`	ErrorPage – nem létező útvonalak kezelése

3.5. Globális állapotkezelés (React Context)

Az alkalmazásban két React Context gondoskodik a globálisan szükséges állapotokról. Ezek a Provider komponensek az App.tsx-ben, a teljes komponensfán belül helyezkednek el, így minden gyermek komponens hozzájuk férhet.

3.5.1. AuthenticatedUserContextProvider

Ez a context tárolja a jelenleg bejelentkezett felhasználó adatait. Az alkalmazás indulásakor (useEffect a mountoláskor) automatikusan lekérdezi a /api/Auth/me endpointot, és ha a localStorage-ben érvényes access token van, betölti a felhasználó adatait.

[context/AuthenticatedUserContextProvider.tsx]

```
type AuthUserType = {
  id: number
  username: string
  role: string
  profileImageUrl: string
}

const AuthenticatedUserContextProvider = (
  props: {children: React.ReactNode}
) => {
  const [authUser, setAuthUser] = useState<AuthUserType>()

  useEffect(() => {
      getUserData().then(data => { setAuthUser(data) })
  }, [])

  return (
      <AuthUserContext.Provider value={authUser ? {authUser} : undefined}>
      {props.children}
      </AuthUserContext.Provider>
  )
}
```

Ha a context értéke undefined (nincs bejelentkezett felhasználó), az összes interakciós funkció (like, mentés, kommentelés) figyelmeztetést jelenít meg. A Navbar komponense is erre a contextre támaszkodik: ha van authUser, a profilmenüt és kijelentkezés gombot mutat; ha nincs, bejelentkezés linket.

3.5.2. MobileContextProvider

Ez a context egyetlen boolean értéket – az isMobile flaget – tesz elérhetővé az egész alkalmazás számára. Az ablak szélességét figyeli resize eseményre: ha kisebb mint 768px, mobile módot jelöl.

[context/MobileContextProvider.tsx]

```
const MobileContextProvider = ({children}: {children: ReactNode}) => {
  const [isMobile, setIsMobile] = useState(window.innerWidth < 768)

  const switchToMobile = () => {
      if (isMobile !== window.innerWidth <= 768)
      setIsMobile(window.innerWidth <= 768)
  }
  window.addEventListener("resize", switchToMobile)

  return (
      <MobileContext.Provider value={{isMobile}}>
      {children}
      </MobileContext.Provider>
  )
}
```

Az isMobile flag alapján a ForYou oldal és a ForYouReel komponens teljesen eltérő elrendezést renderel: asztali gépen kétpaneles nézet (kép + leírás egymás mellett), mobilon csak a képes reel látszik, és a leírás egy külön /description/:id oldalon érhető el. Emellett a ContentUpload oldal is függőlegesen rendeződik el mobilon, vízszintes helyett.

3.6. Oldalak és komponenseik részletesen

3.6.1. Home oldal

A főoldalon a felhasználó áttekintést kap az összes feltöltött receptről. Az oldal három komponensből áll: Navbar, HeadBar és Recipes. Az oldal szintű állapot (searchResults) összeköti a HeadBar keresési logikáját a Recipes megjelenítővel.

[pages/Home.tsx]

```
const Home = () => {
  const [searchResults, setSearchResults] = useState<RecipeType[]>([])

  return (
      <>
      <Navbar/>
      <HeadBar onSearch={setSearchResults}/>
      <Recipes searchResults={searchResults}/>
      </>
  )
}
```

A keresési logika a HeadBar komponensben él: 300ms debounce-szal hívja az API-t, az eredményeket az onSearch callback-en keresztül adja fel a Home komponens state-jébe, ami továbbítja a Recipes komponensnek. Ha a keresőmező üres, a Recipes az összes receptet mutatja (TanStack Query cache-ből); ha van keresési eredmény, csak azt rendereli. Ez a "lifting state up" React minta egy tiszta megvalósítása.

3.6.2. Navbar komponens

A Navbar az alkalmazás állandó navigációs sávja, amely minden oldalon megjelenik. Az AuthenticatedUserContext-et olvassa: bejelentkezett felhasználónál profilképet és felhasználónevet jelenít meg, kijelentkezés opcióval; vendég felhasználónál belépés linket mutat. A NavMiddle alkomponens kezeli az aktív menüpont kiemelését, amelyet a setActive() függvénnyel lehet programozottan is vezérelni – ezt használja például a Login oldal sikeres belépés után, amikor visszairányítja a felhasználót a főoldalra és az első menüpontot állítja aktívra.

3.6.3. ForYou oldal

A ForYou oldal az alkalmazás egyik legösszetettebb komponense. Az URL-ből kiolvasott recept ID alapján lekérdezi az aktuális receptet, valamint a következő és előző receptet is (createNextRecipeQueryOption, createPrevRecipeQueryOption). Ez lehetővé teszi a receptek közötti gördülékeny navigációt anélkül, hogy az egész listát újra le kellene tölteni.

Az oldal két teljesen eltérő elrendezést renderel az isMobile flag alapján:

- Asztali nézet: ForYouReel (bal panel, recept képe és interakciók) + nyilak + Description (jobb panel, részletes leírás) egymás mellett
- Mobil nézet: csak a ForYouReel jelenik meg, a Description egy külön /description/:id oldalon érhető el

Egy fontos részlet: az oldal skeleton (mock) adatot renderel amíg az API válasz megérkezik, így elkerülhető a layout ugrás töltés közben.

```
[pages/ForYou.tsx – adatlekérés és reszponzív renderelés]
```

```
const ForYou = () => {
  const navigate = useNavigate()
  const {isMobile} = useMobileContext()
  const {id} = useParams()

  const {data} = useQuery(createRecipeByIdQueryOption(id as string))
  const recipe: RecipeType = data as RecipeType

  const next = useQuery({
      ...createNextRecipeQueryOption(data?.id as string),
      enabled: !!data
  })
  const prev = useQuery({
      ...createPrevRecipeQueryOption(data?.id as string),
      enabled: !!data
  })

  return (
      <>
      {recipe && next && prev ?
      <div className={styles.forYou}>
      <ForYouReel {...recipe}/>
      { !isMobile &&
             <>
             <div className={styles.arrows}>
             <i className="fa-solid fa-circle-down fa-rotate-180"
                    onClick={() => nextFuncion()}/>
             <i className="fa-solid fa-circle-down"
                    onClick={() => prevFuncion()}/>
             </div>
             <Description {...recipe}/>
             </>
      }
      </div>
      : <div>...skeleton...</div>
      }
      </>
  )
}
```

Az oldal érintéses (touch) navigációt is támogat mobilon: a handleTouchStart, handleTouchMove és handleTouchEnd eseménykezelők horizontális swipe gesztust érzékelnek. Ha a swipe távolsága meghalad egy küszöbértéket, a következő vagy előző receptre navigál.

[pages/ForYou.tsx – swipe navigáció]

```
const [touchStart, setTouchStart] = useState<{ x: number } | null>(null);
const [touchEnd, setTouchEnd] = useState<{ x: number } | null>(null);

const handleTouchEnd = useCallback(() => {
  if (!touchStart || !touchEnd) return;
  const xDiff = touchStart.x - touchEnd.x;

  if (xDiff > 0 && next.data) {
      navigate("/foryou/" + next.data.id)  // balra swipe → következő
  } else if (xDiff < 0 && prev.data) {
      navigate("/foryou/" + prev.data.id)  // jobbra swipe → előző
  }
}, [touchStart, touchEnd, next, prev]);
```

3.6.4. Login oldal

A Login oldal felhasználónév és jelszó alapján hitelesíti a felhasználót. A form állapotát két useState hook kezeli (username, password). A bejelentkezési folyamat:

- Az adatok elküldése a publicAPI loginUser() függvényén keresztül a /api/Auth/login endpointra
- A visszakapott JWT access és refresh token eltárolása a localStorage-ben
- Sikeres bejelentkezés után navigálás a főoldalra, az aktív menüpont visszaállítása, majd window.location.reload() az AuthenticatedUserContextProvider újratöltéséhez

[pages/Login.tsx]

```
const Login = () => {
  const [username, setUsername] = useState<string>("")
  const [password, setPassword] = useState<string>("")
  const navigate = useNavigate()

  const goToHome = () => {
      navigate("/")
      setActive(menuItems[0].name)
      window.location.reload()
  }

  const login = async () => {
      await loginUser(username, password) && goToHome()
  }

  return (
      <div className={styles.myForm}>
      <fieldset className={styles.login}>
      <HomeBtn/>
      <h1>Belépés</h1>
      <label htmlFor="name">
             Felhasználónév<br/>
             <input type="text" id='name' required
             onChange={(e) => setUsername(e.target.value)}/>
      </label>
      <label htmlFor="pass">
             Jelszó<br/>
             <input type="password" id='pass' required
             onChange={(e) => setPassword(e.target.value)}/>
      </label>
      <button className={styles.btn} onClick={login}>Belépés</button>
      <p>Ha még nincsen fiókod: <Link to="/signup">Regisztrálj</Link></p>
      </fieldset>
      </div>
  )
}
```

3.6.5. Signup oldal

A Signup oldal TanStack Query useMutation hookot alkalmaz a regisztrációhoz, amely a createRegisterMutationOption konfigurációt használja. A felhasználónév, e-mail cím és jelszó megadása szükséges. A mutáció a /api/Auth/register endpointot hívja, sikertelen kísérlet esetén Sonner toast értesítést jelenít meg.

[pages/Signup.tsx]

```
const Signup = () => {
  const [username, setUsername] = useState<string>("")
  const [password, setPassword] = useState<string>("")

  const reg = useMutation(createRegisterMutationOption(username, password))

  return (
      <div className={styles.myForm}>
      <fieldset className={styles.login}>
      <HomeBtn/>
      <h1>Regisztráció</h1>
      <label htmlFor="name">
            Felhasználónév<br/>
            <input type="text" id='name' required
            onChange={(e) => setUsername(e.target.value)}/>
      </label>
      <label htmlFor="email">
            E-mail<br/>
            <input type="email" id='email'/>
      </label>
      <label htmlFor="pass">
            Jelszó<br/>
            <input type="password" id='pass' required
            onChange={(e) => setPassword(e.target.value)}/>
      </label>
      <input className={styles.btn} type="submit"
            value="Regisztráció" onClick={() => reg.mutate()}/>
      <p>Ha már van fiókod: <Link to="/login">Lépj be</Link></p>
      </fieldset>
      </div>
  )
}
```

Fontos különbség a Login-hoz képest: míg a Login saját async függvénnyel kezeli a bejelentkezést, a Signup a TanStack Query useMutation hookot alkalmazza, ami automatikusan kezeli a loading állapotot és a hibakezelést, megkönnyítve a visszajelzés megjelenítését.

3.6.6. ContentUpload – recept feltöltő oldal

A recept feltöltő oldal (ContentUpload_) egy komplex form, amely két fő alkomponensből áll: ImageUpload (képfeltöltés) és TextUpload (szöveges adatok, nehézségi szint, feltöltés gomb). Alapból vízszintesen vannak elrendezve; mobil nézetben függőlegesen.

A feltöltési folyamat lépései:

- A felhasználó kitölti az összes kötelező mezőt (borítókép, név, leírás, hozzávalók, nehézségi szint, elkészítési idő)

- Opcionálisan hozzáad címkéket
- A feltöltés gomb megnyomásakor az isPostReady() validátor ellenőrzi a mezőket
- Meglévő címkéknél csak az ID-t tároljuk; új, nem létező címkéket előbb POSToljuk a /api/Cimkek endpointra
- A recept végül FormData-ként kerül elküldésre a /api/Recept/with-image endpointra (multipart/form-data)
- Sikeres feltöltés esetén UploadAlert visszajelzés és automatikus átirányítás a saját receptek oldalra

3.6.7. ErrorPage

Az ErrorPage egy egyszerű hibaoldal, amely a React Router * (wildcard) útvonalához van rendelve. Minden nem létező URL esetén megjelenik, és egy HomeBtn komponensen keresztül visszairányítja a felhasználót a főoldalra. Egyszerű felépítése ellenére fontos szerepet játszik a felhasználói élményben, elkerülve az üres fehér oldalt.

3.7. API kommunikáció

A frontend a backend REST API-val Axios segítségével kommunikál. Az API hívások két külön service fájlba vannak szervezve:

- **publicAPI.ts –** autentikációt nem igénylő hívások (receptek listázása, keresés, bejelentkezés, regisztráció)
- **privateAPI.ts –** autentikációt igénylő hívások (like, mentés, komment, feltöltés). Minden kéréshez automatikusan hozzáadja a JWT tokent az Authorization headerbe

```
[services – JWT token csatolása minden privát kéréshez]
```

```
const API_BASE_URL = process.env.REACT_APP_API_URL;

const privateRequest = async (method, endpoint, data?) => {
  const token = localStorage.getItem("accessToken")
  return await axios({
      method,
      url: `${API_BASE_URL}/${endpoint}`,
      data,
      headers: { Authorization: `Bearer ${token}` }
  })
}
```

3.8. Állapotkezelés (TanStack Query)

A szerverről érkező adatok kezelésére TanStack Query-t (React Query) használtunk. A lekérdezési konfigurációk a queryOptions/ mappában, a mutációk a mutationOptions/ mappában találhatók, így a fetch logika teljesen el van választva a UI komponensektől.

Főbb előnyök:

- Automatikus cache-elés: ugyanazt az adatot nem kéri le többször feleslegesen
- Background refetch: az adatok automatikusan frissülnek, ha a felhasználó visszatér az oldalra
- Loading és error állapotok egyszerű kezelése a komponensekben
- Optimista frissítések lehetősége (pl. like gomb azonnal reagál, majd szinkronizálódik)

[queryOptions/createRecipeQueryOption.ts – példa query konfiguráció]

```
export const createRecipeByIdQueryOption = (id: string) => ({
  queryKey: ["recipe", id],
  queryFn: () => getRecipeById(id),
})

export const createNextRecipeQueryOption = (id: string) => ({
  queryKey: ["recipe", "next", id],
  queryFn: () => getNextRecipe(id),
})

export const createPrevRecipeQueryOption = (id: string) => ({
  queryKey: ["recipe", "prev", id],
  queryFn: () => getPrevRecipe(id),
})
```

3.9. Autentikáció kezelése

A frontend JWT alapú autentikációt használ. A működés lépései:

- A felhasználó belép a Login oldalon
- A backend visszaad egy rövid életű JWT access tokent (15 perc) és egy refresh tokent
- Mindkét token eltárolásra kerül a localStorage-ben
- Minden védett API kérésnél a privateAPI automatikusan hozzáadja az access tokent az Authorization headerhez
- Ha az access token lejár, a /api/Auth/refresh endpoint segítségével új token igényelhető a refresh tokennel
- Az AuthenticatedUserContextProvider az oldal betöltésekor ellenőrzi a token érvényességét, és betölti a felhasználói adatokat

3.10. Stíluskezelés (CSS Modules)

A stílusok kezelésére CSS Modules technológiát alkalmaztunk. Minden komponens saját .module.css fájllal rendelkezik, amelyre a komponens importként hivatkozik. A build folyamat során a CSS osztálynevekhez egyedi azonosítók kerülnek, így teljesen ki van zárva a globális névütközés más komponensek stílusaival.

[Példa CSS Module használat]

```
// ForYou.module.css
.forYou {
  display: flex;
  flex-direction: row;
  height: 100vh;
}

// ForYou.tsx
import styles from '../css/ForYou.module.css'
<div className={styles.forYou}>...</div>
```

3.11. TypeScript típusdefiníciók

A TypeScript típusok a types/ mappában találhatók, és biztosítják, hogy a komponensek mindig a helyes adatstruktúrával dolgozzanak. A backend API válaszainak típusai itt vannak definiálva, ami fordítási időben korai hibadetektálást tesz lehetővé.

[types/RecipeTypes.ts – RecipeType típusdefiníció]

```
export type RecipeType = {
  id: string
  nev: string
  leiras: string
  hozzavalok: string
  elkeszitesiIdo: number
  nehezsegiSzint: string
  likes: number
  likeolvaVan: boolean
  mentveVan: boolean
  kepUrl: string
  feltoltoUsername: string
  cimkek: CimkeType[]
}
```

3.12. Hibakezelés és felhasználói visszajelzés

A felhasználói élmény javítása érdekében külön figyelmet fordítottunk a hibakezelésre:

- Sonner toast értesítések sikeres és sikertelen műveleteknél egyaránt (pl. sikeres feltöltés, hibás bejelentkezés)
- ErrorPage komponens nem létező route-ok esetén, visszanavigálási lehetőséggel
- Loading állapotok kezelése adatlekérés közben (TanStack Query isLoading flag)
- Skeleton/mock adat renderelése az API válasz megérkezéséig (ForYou oldal)
- Validáció a ContentUpload oldalon a feltöltés előtt (isPostReady() függvény)

3.13. Felhasználói élmény (UI/UX)

A frontend kialakításánál fontos szempont volt a modern és intuitív felhasználói élmény:

- Reszponzív design: a MobileContextProvider alapján teljesen eltérő elrendezés mobilon és asztali nézetben
- Swipe navigáció: a ForYou oldalon érintéses gesztussal lehet receptek között váltani
- Debounce a keresésnél: 300ms késleltetés, így nem indul el felesleges API hívás minden leütésnél
- TanStack Query cache: a már megtekintett adatok azonnal betöltődnek ismételt látogatásnál
- CSS Modules: komponens szintű izoláció, egységes vizuális megjelenés

3.14. Weboldal elérése

A frontend indításához Visual Studio Code-ban nyisd meg a projekt frontend/ mappáját, majd a terminálba írd be:

```
npm install
npm run dev
```

A feltüntetett linken (alapértelmezetten http://localhost:5173) megtekinthető az oldal. A szerver leállítása a terminálban Ctrl + C billentyűkombinációval lehetséges. A backend szerver futtatása szükséges az adatok eléréséhez (lásd 2.7. fejezet).

4. Összegzés

A projekt során sikeresen megvalósítottunk egy modern, biztonságos és skálázható backend API-t egy recept megosztó webalkalmazáshoz. Az ASP.NET Core és Entity Framework Core használatával egy jól strukturált, könnyen karbantartható kódbázist hoztunk létre.

A legfontosabb eredmények:

- RESTful API 36+ endpoint-tal
- JWT alapú authentikáció és authorization
- Cloud alapú képkezelés ImageKit-tel
- Komplex keresési és szűrési lehetőségek
- Admin funkciók a tartalom moderálásához
- Interaktív API dokumentáció Scalar-ral

A projekt során értékes tapasztalatokat szereztünk a modern backend fejlesztésben, különösen a biztonságos authentikáció, adatbázis tervezés és külső szolgáltatások integrációja terén.

4.1. Továbbfejlesztési lehetőségek

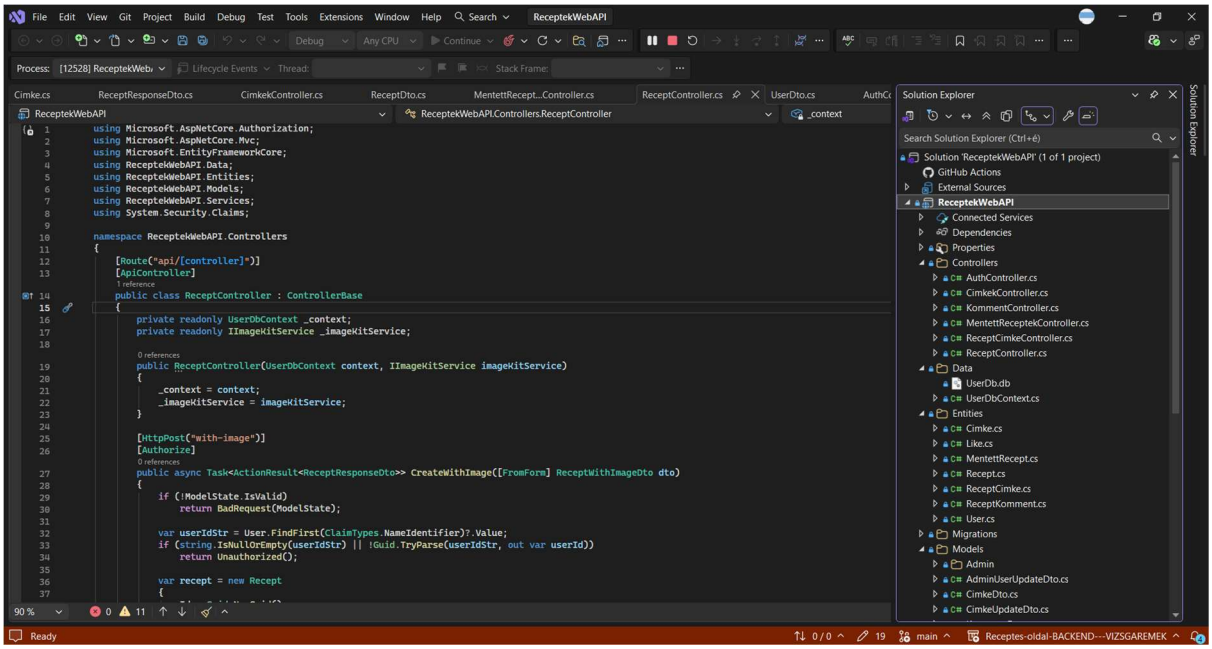
Az alkalmazás további fejlesztési lehetőségei:

- **Értékelési rendszer:** 5 csillagos értékelés receptekhez
- **Ajánló algoritmus:** Személyre szabott recept ajánlások a felhasználók érdeklődése alapján
- **Bevásárlólista generálás:** Automatikus bevásárlólista készítés kiválasztott receptekből
- **Étkezési tervező:** Heti menü tervezés funkcionalitás
- **Social funkciók:** Felhasználók követése, közösségi hírcsatorna
- **Táplálkozási információ:** Kalória és tápanyag számítás receptekhez
- **Többnyelvűség:** Nemzetközi felhasználók támogatása
- **Mobilalkalmazás:** Native mobil appok iOS-re és Android-ra
- **Adatbázis migráció:** SQLite-ról PostgreSQL vagy MS SQL Server-re váltás nagyobb skálázhatóságért
- Meglévő funkciók további optimalizációja.

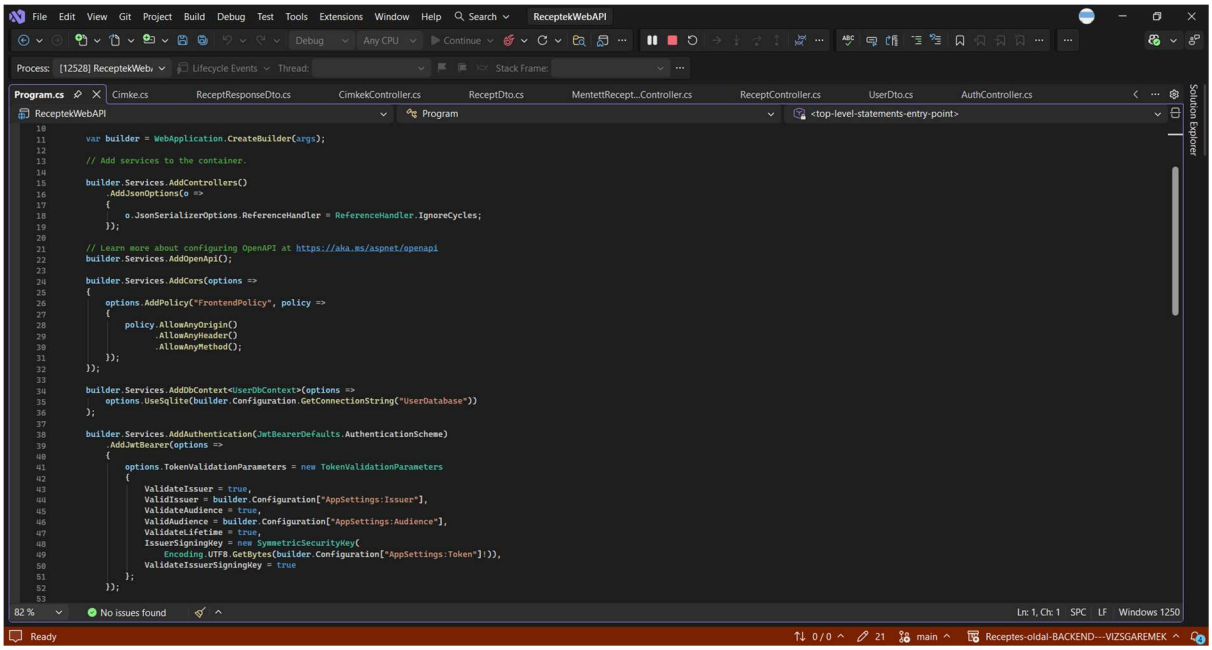
5. Melléklet - Képernyőképek

5.1. Fejlesztői környezet

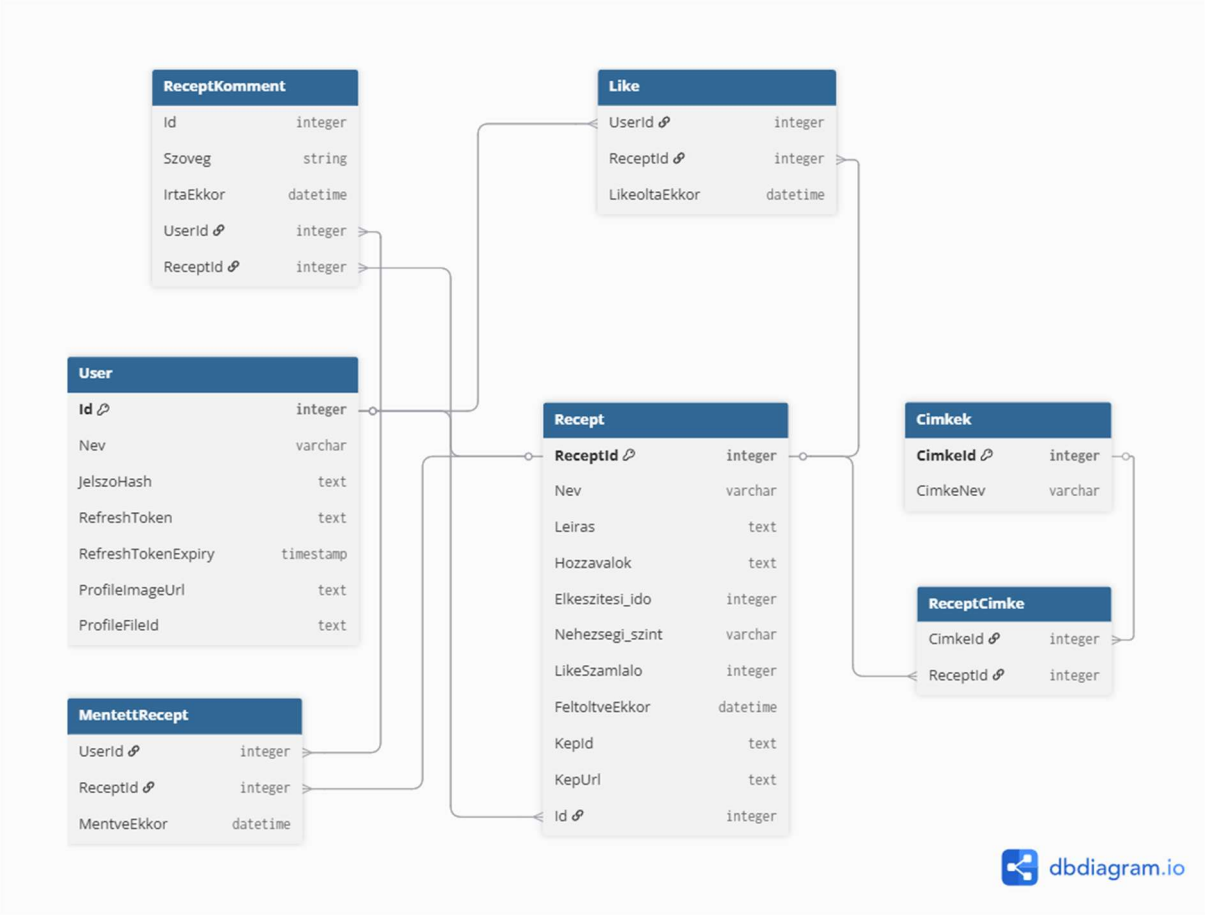
Solution Explorer



Program.cs fájl

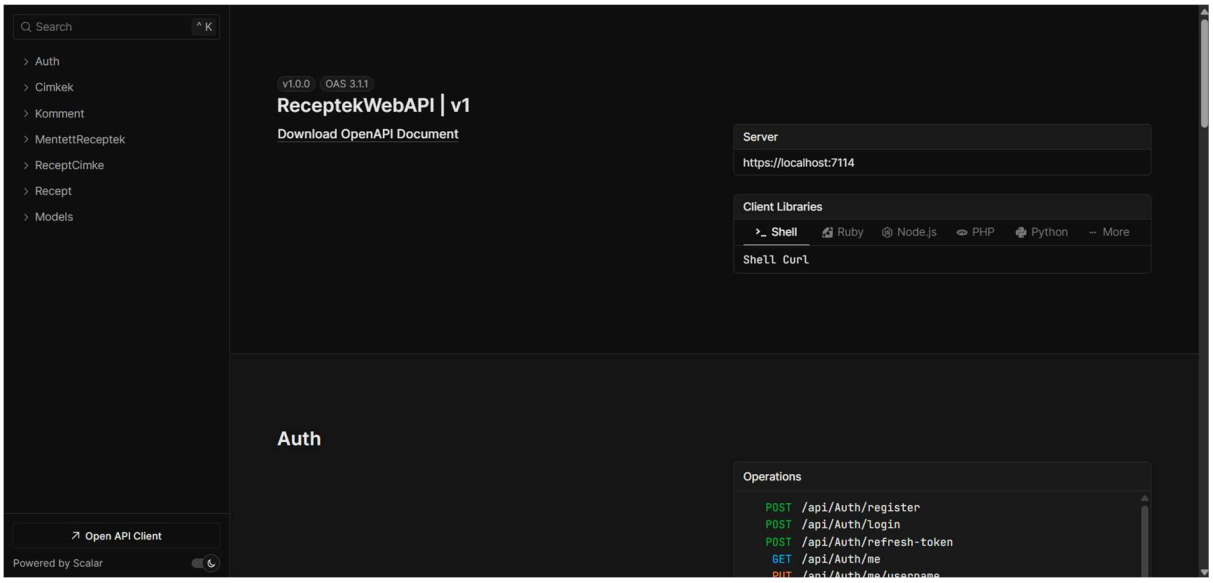


5.2. Adatbázis diagram

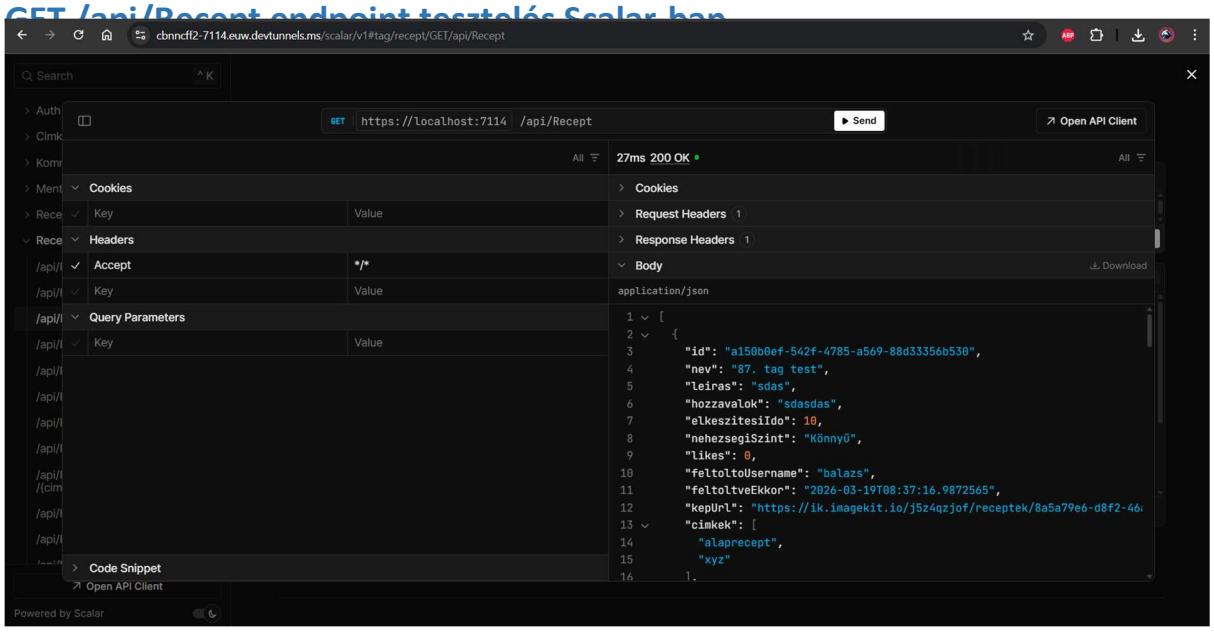


5.3. API dokumentáció és tesztelés

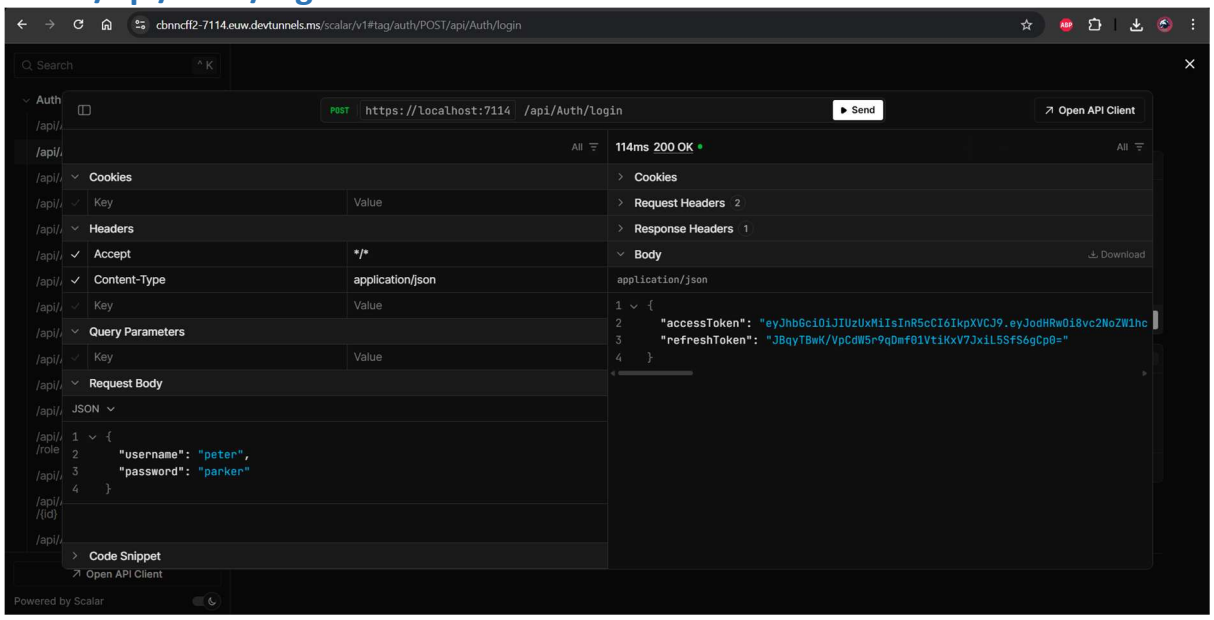
Scalar API dokumentáció főoldal



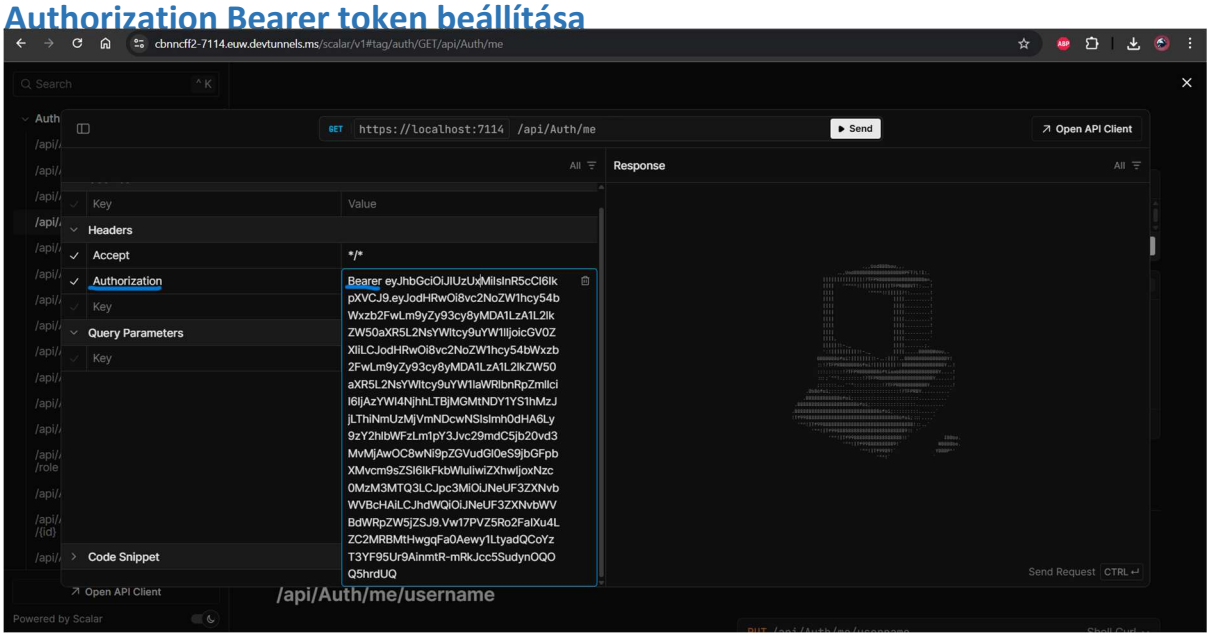
GET /api/Recept endpoint tesztelés Scalar-ban



POST /api/Auth/login sikeres válasz

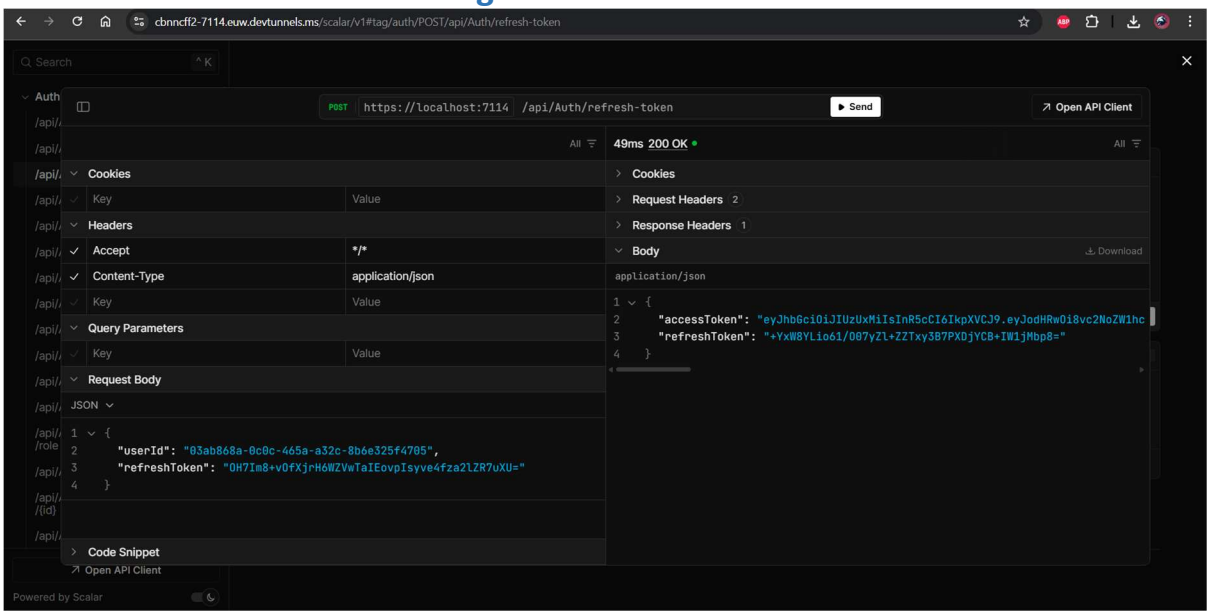


Authorization Bearer token beállítása

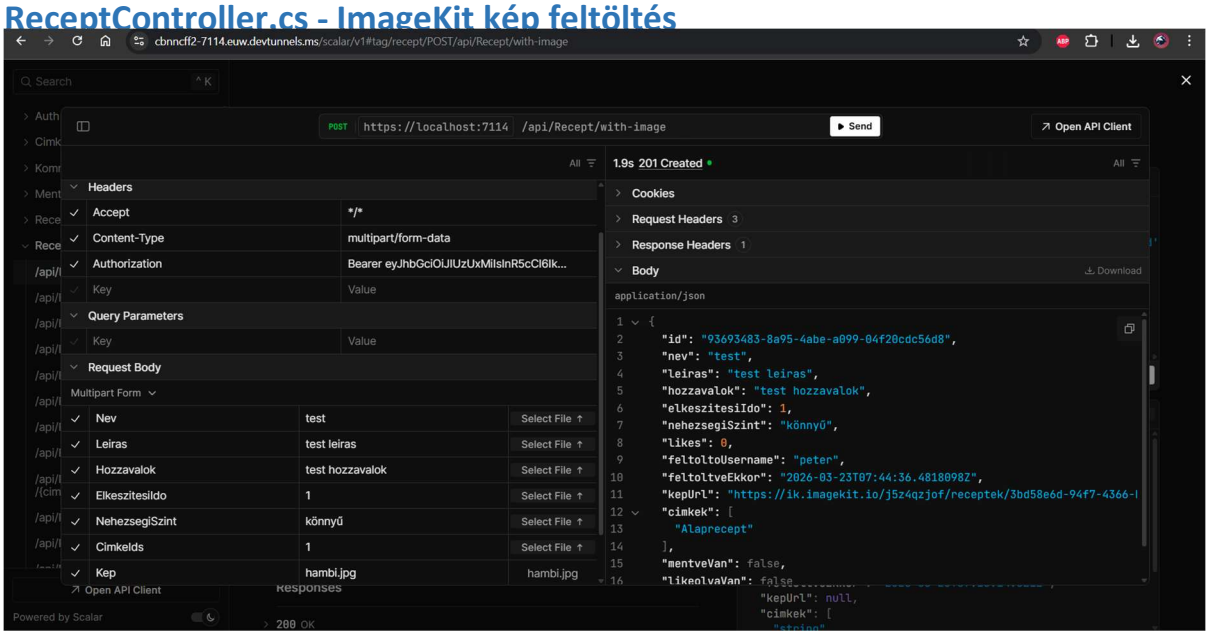


5.4. Kódrészletek

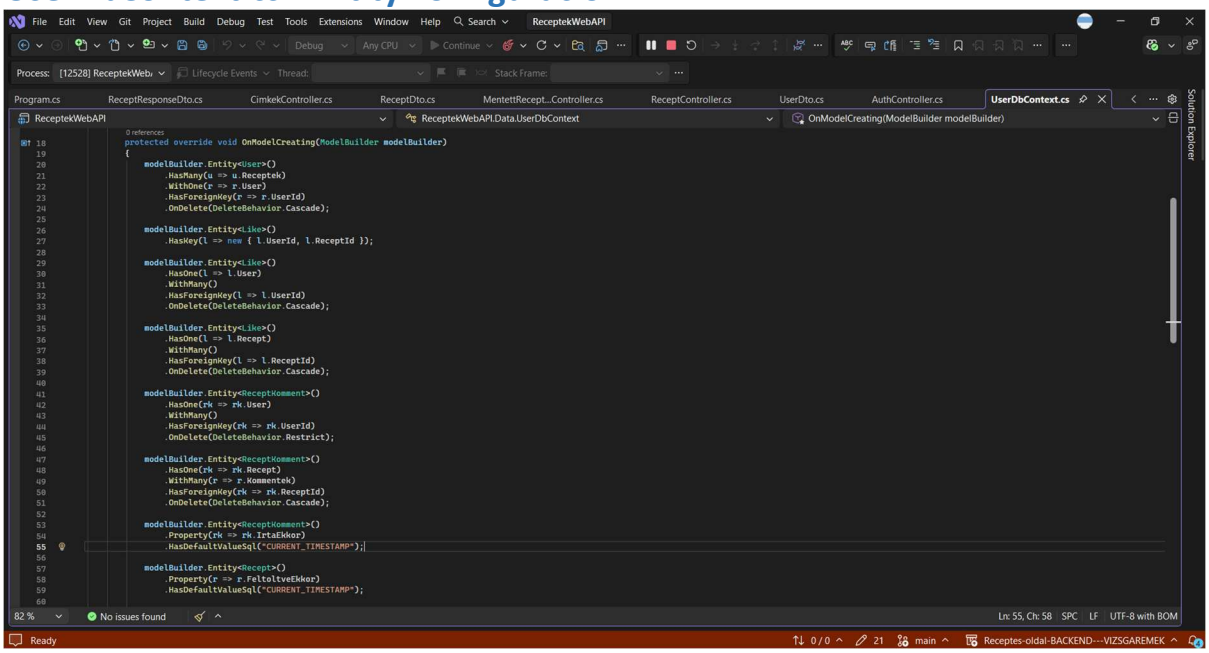
AuthController.cs - JWT token generálás



ReceptController.cs - ImageKit kép feltöltés



UserDbContext.cs - Entity konfiguráció



6. Irodalomjegyzék

- Microsoft Docs - ASP.NET Core documentation: https://docs.microsoft.com/aspnet/core/
- Entity Framework Core dokumentáció: https://docs.microsoft.com/ef/core/
- JWT.io - Introduction to JSON Web Tokens: https://jwt.io/introduction
- ImageKit dokumentáció: https://docs.imagekit.io/
- RESTful API Design - Best Practices: https://restfulapi.net/
- Scalar dokumentáció: https://github.com/scalar/scalar
- React dokumentáció: https://react.dev/learn
- TypeScript dokumentáció: https://www.typescriptlang.org/docs/
- GitHub repository: https://github.com/KBencee/Receptes-oldal-BACKEND---VIZSGAREMEK
  https://github.com/KBencee/Receptes-oldal-FRONTEND---VIZSGAREMEK
  https://github.com/KBencee/Receptes-oldal---VIZSGAREMEK